

Virtual Memory

- **Definition:** A memory management technique that allows the execution of processes that are not entirely in main memory. It separates user logical memory from physical memory.
- **Benefits:**
 - Allows larger programs than physical memory.
 - Allows more programs to run concurrently.
 - Frees programmers from memory concerns.
- **Address Space:**
 - **Logical Address Space:** The set of all logical addresses generated by a program.
 - **Physical Address Space:** The set of all physical addresses corresponding to the logical addresses.

Demand Paging

- **Concept:** Pages are loaded into memory only when they are needed (demanded).
- **Page Fault:** An event that occurs when a program tries to access a page that is not currently in physical memory.
 1. CPU generates logical address.
 2. MMU translates to physical address; invalid bit or page not in memory trap.
 3. OS handles page fault:
 - Determine if the reference was valid/invalid.
 - If invalid, terminate process.
 - If valid but page not loaded, find a free frame.
 - Schedule disk operation to read page into frame.
 - When I/O complete, reset page table entry and restart instruction.
- **Performance:** Effective Access Time (EAT) = $(1 - p) \times \text{memory access} + p \times (\text{page fault overhead} + \text{swap page out} + \text{swap page in} + \text{restart overhead})$, where p is page fault rate.

Page Replacement Algorithms

When a page fault occurs and no free frames are available, a page must be selected for replacement.

- **FIFO (First-In, First-Out):** Replaces the oldest page in memory. Simple but can suffer from Belady's Anomaly (more frames $\rightarrow$ more page faults).
- **Optimal (OPT/MIN):** Replaces the page that will not be used for the longest period of time. Impossible to implement in practice but serves as a benchmark.
- **LRU (Least Recently Used):** Replaces the page that has not been used for the longest period of time.
 - **Implementation:** Counter or Stack.
 - Good approximation of OPT.
- **Approximation Algorithms:**
 - **Second-Chance (Clock):** FIFO with a reference bit. If bit is 1, give it a second chance (clear bit and move to end of FIFO queue).
 - **LFU (Least Frequently Used):** Replaces the page with the smallest count.
 - **MFU (Most Frequently Used):** Replaces the page with the largest count (based on idea that page with smallest count was probably just brought in and has yet to be used).

Allocation of Frames

- **Minimum Number of Frames:** Determined by the architecture (e.g., number of pages an instruction can touch).
- **Allocation Schemes:**
 - **Fixed Allocation:**
 - **Equal Allocation:** Distribute m frames equally among n processes (m/n frames per process).
 - **Proportional Allocation:** Allocate frames based on process size ($s_i / \sum s_j \times m$).
 - **Priority Allocation:** Use proportional allocation but based on priority rather than size.
 - **Global Replacement:** A process can select a replacement frame from the set of all frames, even if that frame is currently allocated to another process.
 - Higher throughput.
 - Process's page fault rate can vary.
 - **Local Replacement:** Each process selects a replacement frame only from its own set of allocated frames.
 - More consistent per-process performance.
 - Underutilized frames may exist.

Thrashing

- **Definition:** A state where a process spends more time paging than executing, leading to very low CPU utilization.
- **Cause:** When a process does not have enough frames to hold its working set, it continuously pages in and out, resulting in a high page fault rate.
- **Working Set Model:**
 - **Working Set:** The set of pages actively used by a process over a fixed time window Δ .
 - **Goal:** Allocate enough frames to each process to hold its working set.
- **Solutions:**
 - If CPU utilization is too low and paging is high, suspend some processes to free up frames.
 - Increase the amount of physical memory.
 - Reduce the degree of multiprogramming.

Demand Segmentation

- **Concept:** Segments (logical units of a program) are brought into memory only when needed, similar to demand paging.
- **Benefits:** Combines the benefits of segmentation (user view of memory, protection) with demand paging (efficient memory use, larger programs).
- **Challenges:** Segment size variability can lead to external fragmentation.

Cache Memory Organization

- **Definition:** A smaller, faster memory that stores copies of data from frequently used main memory locations.
- **Locality of Reference:**
 - **Temporal Locality:** Recently accessed items are likely to be accessed again soon.
 - **Spatial Locality:** Items whose addresses are near one another tend to be referenced close together in time.
- **Mapping Functions:**
 - **Direct-Mapped:** Each block from main memory maps to exactly one cache line. Simple but high conflict misses.
 - **Fully Associative:** Any block from main memory can be placed in any cache line. Flexible but complex to implement.
 - **Set-Associative:** A compromise; each block maps to a specific set of cache lines, and within that set, it can be placed in any line.
- **Write Policies:**
 - **Write-Through:** Writes data to both cache and main memory simultaneously. Simpler, but slower writes.
 - **Write-Back:** Writes data only to cache, marks line as "dirty." Main memory is updated only when the dirty line is replaced. Faster writes, but complex.
- **Cache Hits/Misses:**
 - **Hit:** Data found in cache.
 - **Miss:** Data not found in cache, must fetch from main memory.
 - **Hit Rate:** Percentage of accesses found in cache.
 - **Miss Rate:** Percentage of accesses not found in cache ($1 - \text{Hit Rate}$).

Role of Operating System in Security

- **Protection:** Mechanisms to control access of programs, processes, or users to resources defined by the computer system.
- **Security:** Defends the system against internal and external attacks.
- **Key OS Functions:**
 - **Authentication:** Verifying the identity of a user or process (e.g., passwords).
 - **Authorization:** Determining what an authenticated user or process is permitted to do.
 - **Access Control:** Enforcing authorization rules (e.g., Access Control Lists, Capability Lists).
 - **Auditing/Logging:** Recording security-relevant events for later review.
 - **Resource Protection:** Ensuring proper memory, CPU, file, and device access.
 - **Kernel Protection:** Isolating the OS kernel from user processes.

Security Breaches

- **Malware:** Malicious software.
 - **Viruses:** Attaches to legitimate programs; requires user action to spread.
 - **Worms:** Self-replicating, self-propagating without user action.
 - **Trojan Horses:** Disguised as legitimate software.
 - **Ransomware:** Encrypts data and demands payment for decryption key.
 - **Spyware:** Gathers information without user knowledge.
- **Denial of Service (DoS/DDoS):** Flooding a system with requests to make it unavailable.
- **Phishing:** Attempting to acquire sensitive information by masquerading as a trustworthy entity.
- **Man-in-the-Middle (MitM):** Attacker intercepts communication between two parties.
- **Buffer Overflow:** Writing beyond allocated buffer memory, potentially overwriting other data or injecting malicious code.
- **Rootkit:** Collection of tools that enable administrator-level access to a computer or network without being detected.

System Protection

- **Domain of Protection:** Defines the set of resources that a process can access.
 - **Access Matrix:** A conceptual model for protection domains.
- | Objects $\rightarrow$ | F_1 | F_2 | P_1 |
|-----------------------|-------|-------|--------|
| D_1 | read | read | switch |
| D_2 | write | | |
- **Implementation of Access Matrix:**
 - **Global Table:** Stores (domain, object, rights) triplets. Large and inefficient.
 - **Access Lists (ACLs) for Objects:** Each object has a list of (domain, rights) pairs. Easier to manage object access.
 - **Capability Lists for Domains:** Each domain has a list of (object, rights) pairs. Easier to manage domain capabilities.
 - **Revocation of Access Rights:** Challenging, especially with capability lists.

Password Management

- **Hashing:** Storing one-way hashes of passwords instead of plain text.
 - **Salting:** Adding a random, unique string to each password before hashing to prevent rainbow table attacks.
- **Strong Passwords:** Encourage users to create long, complex passwords.
- **Multi-Factor Authentication (MFA):** Requires two or more verification methods (e.g., password + one-time code).
- **Password Policies:**
 - Minimum length.
 - Character complexity (uppercase, lowercase, numbers, symbols).
 - Expiration periods.
 - History (prevent reuse of old passwords).
- **Secure Storage:** Passwords (even hashed) should be stored securely, often encrypted.
- **Authentication Protocols**